Apply **Computational Thinking** to LAB211 Project

FALL26

Mô hình triển khai LAB211

Fall 26

Project Base Learning

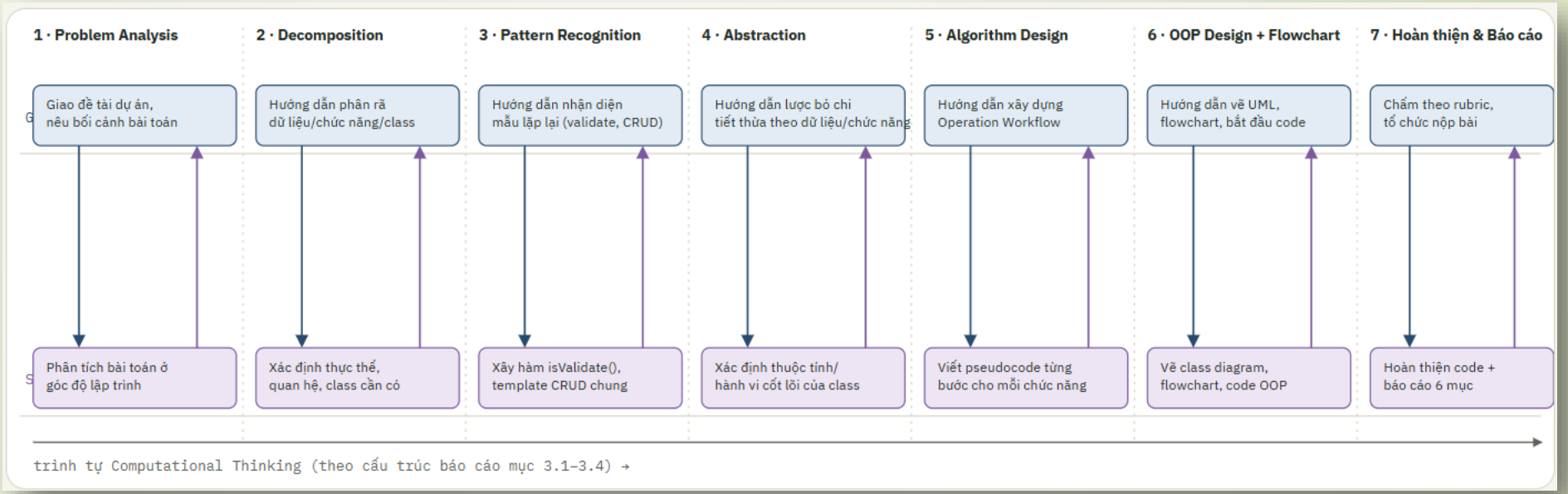
- PBL = **2 dự án OOP =>** rèn luyện dựa trên | | **4 thành phần Computational Thinking**
- Rubric 100 điểm:
  - Functional and Data **Recognition** **20đ**
  - Apply **Object-Oriented Programming** (OOP) to the project **40đ**
  - Apply **C**omputational **T**hinking to **Problem Analysis** **30đ**
    (*Decomposition 8, Pattern Recognition 6, Abstraction 8, Algorithm Design 8*)
  - Code style & Report **10đ**

```
PBL = 2 dự án OOP rèn luyện qua 4 thành phần Computational Thinking
```

Computational **T**hinking to **Problem Analysis**

- **DECOMPOSITION**
  Phân rã bài toán theo dữ liệu, theo chức năng, theo class (OOP view).
- **PATTERN RECOGNITION**
  Nhận diện mẫu lặp lại: quy luật validate dữ liệu, thao tác CRUD chung cho các thực thể.
- **ABSTRACTION**
  Trừu tượng hóa theo dữ liệu (*thuộc tính cốt lõi*) và theo chức năng (*mỗi chức năng là 1 bài toán nhỏ độc lập*).
- **ALGORITHM DESIGN**
  Xác định các bước xử lý cho từng chức năng, xây dựng Operation Workflow/Flowchart (pseudocode) trước khi triển khai mã nguồn.

MÔ HÌNH GIẢNG VIÊN – SINH VIÊN THEO GIAI ĐOẠN



Computational Thinking

Mục tiêu

Trang bị cho sinh viên năng lực tư duy giải quyết vấn đề, tư duy logic và khả năng giao tiếp kỹ thuật theo hướng **Computational Thinking** (CT), giúp:

- Phân tích, mô hình hóa, thiết kế và trình bày giải pháp rõ ràng cho con người và máy tính.
- Sử dụng các AI tools và phần mềm hỗ trợ như là phương tiện triển khai và kiểm chứng.

Mục tiêu

- CT là một phương pháp sư phạm nhằm rèn luyện tư duy giải quyết vấn đề cho người học thông qua cách tiếp cận của khoa học máy tính
- 4 thành phần quan trọng của CT
  - **Decomposition** – Phân tích bài toán thành các phần nhỏ dễ xử lý.
  - **Pattern recognition** – Nhận diện mô hình và tính tương đồng để hướng tới áp dụng và sử dụng giải pháp nhiều hơn một lần.
  - **Abstraction** – Khái quát hoá vấn đề cốt lõi để tạo giải pháp có tính tổng quát
  - **Algorithms** – Xây dựng quy trình giải quyết từng phần nhỏ, một cách tuần tự

Apply to LAB211 Project

Bối cảnh của bài toán

Bài toán thực tế

- Công ty bảo hiểm muốn số hoá việc quản lý xe + hợp đồng bảo hiểm, dữ liệu lưu trong file nhị phân, xử lý bằng Java Console + OOP.

Bài toán phân tích theo góc độ lập trình

- Xây dựng chương trình ở mức trừu tượng, thấy được các công việc cần làm bao gồm:

1. Đọc/ghi file nhị phân (*carInfo.dat, insurances.dat*)
2. Quản lý **2 thực thể chính**: **Car (Vehicle) – InsuranceStatement**
3. Áp dụng kỹ thuật lập trình:
   - OOP
   - Validation chặt chẽ (*Đảm bảo chỉ chấp nhận dữ liệu đúng, hợp lệ*)
   - Thống kê & báo cáo

Áp dụng – Computational Thinking

1 - Decomposition – *Phân rã bài toán*

1.1 Phân rã theo dữ liệu (**Data Decomposition**).

File	Thực thể	Vai trò
carInfo.dat	Car	Lưu trữ thông tin về xe (*biển số, chủ xe, hãng, giá trị, ngày đăng ký, nơi đăng ký, loại xe*)
insurances.dat	InsuranceStatement	Lưu trữ thông tin hợp đồng bảo hiểm (*mã BH, ngày lập, biển số, khách hàng, kỳ hạn, phí*)

- **Quan hệ (Relationship)**
  - 1 **Car** → nhiều **InsuranceStatement**
    (một xe có thể được bảo hiểm nhiều lần / nhiều năm)
  - **InsuranceStatement** phụ thuộc **Car**
    (thông qua license plate)
- **Quản lý dữ liệu**
  - Đọc/ghi từ file: **carInfo.dat và insurances.dat**
  - Save changes khi chọn "**Save data**" hoặc khi thoát chương trình
    (*nếu dữ liệu đã bị thay đổi*)

Áp dụng – Computational Thinking

1 - Decomposition – *Phân rã bài toán*

1.2 Phân rã theo chức năng (**Functional Decomposition**).

Nhóm	Chức năng
File I/O	Read / Write binary data (Save data, Load data)
Car	Add, Find, Update, Delete, Display
Insurance	Add, List theo năm (sort), Report uninsured cars (sort)
Validation	License plate unique, date hợp lệ, value > 999, vehicle type ∈ {5,7,9}, insurance period ∈ {12,24,36}
UI	Menu, Table display, Sort ASC/DESC

Áp dụng – Computational Thinking

1 - Decomposition – *Phân rã bài toán*

1.3 Phân rã theo class (**OOP View**).

- **Các lớp cần có trong chương trình**
  - Car (Vehicle)
  - InsuranceStatement
  - CarManager
  - InsuranceManager
  - FileUtils
  - ValidationUtils
  - Menu

Áp dụng – Computational Thinking

2 - Pattern Recognition – *Nhận diện mẫu*

- Nhận diện dựa trên quy luật lặp lại

Thực thể	Pattern	Đặc điểm
Car (license plate)	50H3-123.45	2 số + 1 chữ + 1 số, gạch ngang, 3 số, dấu chấm, 2 số
InsuranceStatement (insurance id)	0006	Chuỗi số gồm 4 ký tự

- Giải pháp:
  - Sử dụng Regular Expression
  - Xây dựng hàm ***isValidate*** (…)

Áp dụng – Computational Thinking

2 - Pattern Recognition – *Nhận diện mẫu*

- Nhận diện dựa trên thao tác dữ liệu(CRUD)

Thao tác	Car	Insurance
List	✓	✓ (report theo năm)
Add	✓	✓
Search	✓	✗
Update	✓	✗
Remove	✓	✗

- Tạo **template xử lý CRUD** chung cho các thực thể (Riêng Insurance chỉ cần Add + List, phục vụ báo cáo thống kê)

Áp dụng – Computational Thinking

3 - Abstraction – *Trừu tượng hóa bài toán*

- Xác định thuộc tính & hành vi cơ sở, bỏ bớt các chi tiết không cần thiết. Từ đó tìm được những thuộc tính nào cần thiết cho việc mô tả thông tin, những hành vi cơ sở của đối tượng tương ứng
- Trừu tượng hóa đối với dữ liệu
  - **Car** : license plate, car owner, car brand, value, registration date, registration place, vehicle type.
  - **InsuranceStatement** : insurance id, established date, license plate, customer name, insurance period, insurance fees.

Kết quả:
Xác định được các class cần thiết cho việc mô hình hóa dữ liệu và xử lý dữ liệu theo yêu cầu của các chức năng cần có trong chương trình

Áp dụng – Computational Thinking

3 - Abstraction – *Trừu tượng hóa bài toán*

- Trừu tượng hóa theo chức năng
  - **Mỗi chức năng được xem là một bài toán nhỏ, độc lập trong chương trình**
  - **Ví dụ: chức năng "Add a car information"**
    - Không cần biết menu
    - Chỉ cần quan tâm đến xử lý: validate → add → message

Kết quả:
Xác định được các class cần thiết cho việc mô hình hóa dữ liệu và xử lý dữ liệu theo yêu cầu của các chức năng cần có của chương trình

Áp dụng – Computational Thinking

4 - Algorithm Design – *Thiết kế thuật toán*

- Xác định các bước xử lý cho chức năng chính cần thực hiện, từ đó đưa ra "**Operation Workflow**"
- **Ví dụ: Add a car information**

  Input: Car info

  Output: Success / Failure

  Process:

  - Step 1: Nhập dữ liệu
  - Step 2: Validate từng field (*không null, biển số duy nhất, chủ xe 2-35 ký tự, ngày hợp lệ, giá trị* > 999, vehicle type ∈ {5,7,9})
  - Step 3: Nếu hợp lệ → add vào danh sách xe
  - Step 4: Thông báo kết quả

Kết quả:
Xác định được **pseudocode một cách tường minh** trước khi implement bằng code Java.

Triển khai CT

Computational Thinking	Công việc cần thực hiện trong project
Decomposition	Chia lớp Car / InsuranceStatement, chia manager
Pattern Recognition	License plate, Insurance ID, Date, CRUD
Abstraction	Class model, Utils
Algorithm	Flow add / update / delete / report

Cấu trúc báo cáo cần thực hiện

1. Introduction
2. Problem Analysis
3. Computational Thinking
   - 3.1 Decomposition
   - 3.2 Pattern Recognition
   - 3.3 Abstraction
   - 3.4 Algorithm Design
4. OOP Design (UML)
5. Flowchart
6. Conclusion

Rubric đánh giá

Các mục tiêu chính cần thực hiện

Sinh viên cần triển khai project với 5 tiêu chí chính,
(Đánh giá dựa trên thang 100 điểm)

- **Nhận diện các chức năng cơ bản** **20 đ**
- **Áp dụng kiến thức OOP** **40 đ**
- **Áp dụng Computational Thinking** **30 đ**
- **Code style và báo cáo** **10 đ**

1 - Nhận diện chức năng cơ bản (20đ)

Tiêu chí	Điểm	Mô tả
Thêm thông tin xe (validation đúng)	3	Ràng buộc biển số, chủ xe, giá trị, loại xe, …
Tìm kiếm xe theo biển số	2	Xử lý biển số không tồn tại
Cập nhật thông tin xe	2	Cập nhật đúng, không cho sửa biển số
Xóa thông tin xe	2	Không xóa được nếu đã đăng ký bảo hiểm, có xác nhận
Thêm hợp đồng bảo hiểm (validation đúng)	3	Kiểm tra mã BH duy nhất, kỳ hạn, tính phí đúng công thức
Danh sách hợp đồng bảo hiểm theo năm (sort)	3	Đúng năm, đúng field sort, đúng thứ tự
Báo cáo xe chưa đăng ký bảo hiểm (sort)	3	Đúng field sort, đúng thứ tự
Lưu / Load dữ liệu (file nhị phân)	2	Thông báo thành công / thất bại

2 - Áp dụng kiến thức OOP (40đ)

Tiêu chí	Điểm	Mô tả
Định nghĩa class rõ ràng (Car, InsuranceStatement, …)	10	Thuộc tính + phương thức đúng
Encapsulation (private + getter/setter)	6	Không cho phép truy xuất dữ liệu trực tiếp
Constructor, toString(), equals() override	8	Áp dụng hợp lý
Kế thừa / interface / polymorphism	16	Có ít nhất 1 áp dụng nâng cao

3 - Áp dụng Computational Thinking (30đ)

Tiêu chí	Điểm	Mô tả
Decomposition	8	Tách thành module hợp lý (Car, Insurance, File…)
Pattern Recognition	6	Nhận diện điểm chung giữa các thực thể
Abstraction	8	Loại bỏ chi tiết thừa, giữ thuộc tính cốt lõi
Algorithm Design	8	Pseudocode / mô tả bước xử lý trước khi code

5 - Code style và báo cáo (10đ)

Tiêu chí	Điểm	Mô tả
Đặt tên biến, class theo chuẩn	4	CamelCase, PascalCase
Code rõ ràng, có comment	4	Tránh code spaghetti
Báo cáo ngắn gọn, dễ hiểu	2	Có mô tả cách chạy chương trình

Code **Spaghetti** là gì?

Hãy hình dung một đĩa mì **spaghetti**: các sợi mì ***đan xen***, ***chồng chéo***, **không biết đầu nào nối với đầu nào**.

Trong lập trình cũng tương tự. **Code spaghetti** thường có các đặc điểm:

- Một method/class làm ***quá nhiều việc***.
- if/else *lồng nhau quá nhiều cấp*.
- Code ở nhiều nơi ***phụ thuộc chặt chẽ*** *vào nhau*.
- Logic ***xử lý***, truy cập database và hiển thị giao diện ***trộn lẫn và đan xen***.
- Một thay đổi nhỏ có ***nguy cơ làm hỏng nhiều phần khác***.
- Rất ***khó đọc và debug***.



Example



```java
public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Nhap ten: ");
        String name = sc.nextLine();

        System.out.print("Nhap diem Toan: ");
        double math = sc.nextDouble();

        System.out.print("Nhap diem Van: ");
        double literature = sc.nextDouble();

        System.out.print("Nhap diem Anh: ");
        double english = sc.nextDouble();

        double average = (math + literature + english) / 3;

        if (average >= 8) {
            System.out.println("Ten: " + name);
            System.out.println("Diem TB: " + average);
            System.out.println("Xep loai: Gioi");
        } else {
            if (average >= 6.5) {
                System.out.println("Ten: " + name);
                System.out.println("Diem TB: " + average);
                System.out.println("Xep loai: Kha");
            } else {
                if (average >= 5) {
                    System.out.println("Ten: " + name);
                    System.out.println("Diem TB: " + average);
                    System.out.println("Xep loai: Trung binh");
                } else {
                    System.out.println("Ten: " + name);
                    System.out.println("Diem TB: " + average);
                    System.out.println("Xep loai: Yeu");
                }
            }
        }

        sc.close();
    }
```

1

2

3